

DCC192

2025/2



Desenvolvimento de Jogos Digitais

A7: Física I – Objetos Rígidos

Prof. Lucas N. Ferreira

Plano de Aula



- ▶ Física em Jogos Digitais
- ▶ Movimentação de Objetos Rígidos
 - ▶ Método de Euler Semi-Implicito
- ▶ Aceleração da gravidade
- ▶ Atrito
- ▶ Resistência do Meio

Física em Jogos Digitais



Geralmente, em jogos digitais queremos **mover objetos rígidos** por meio de aplicação de **forças** causadas pelo jogador ou por outros objetos do jogo:



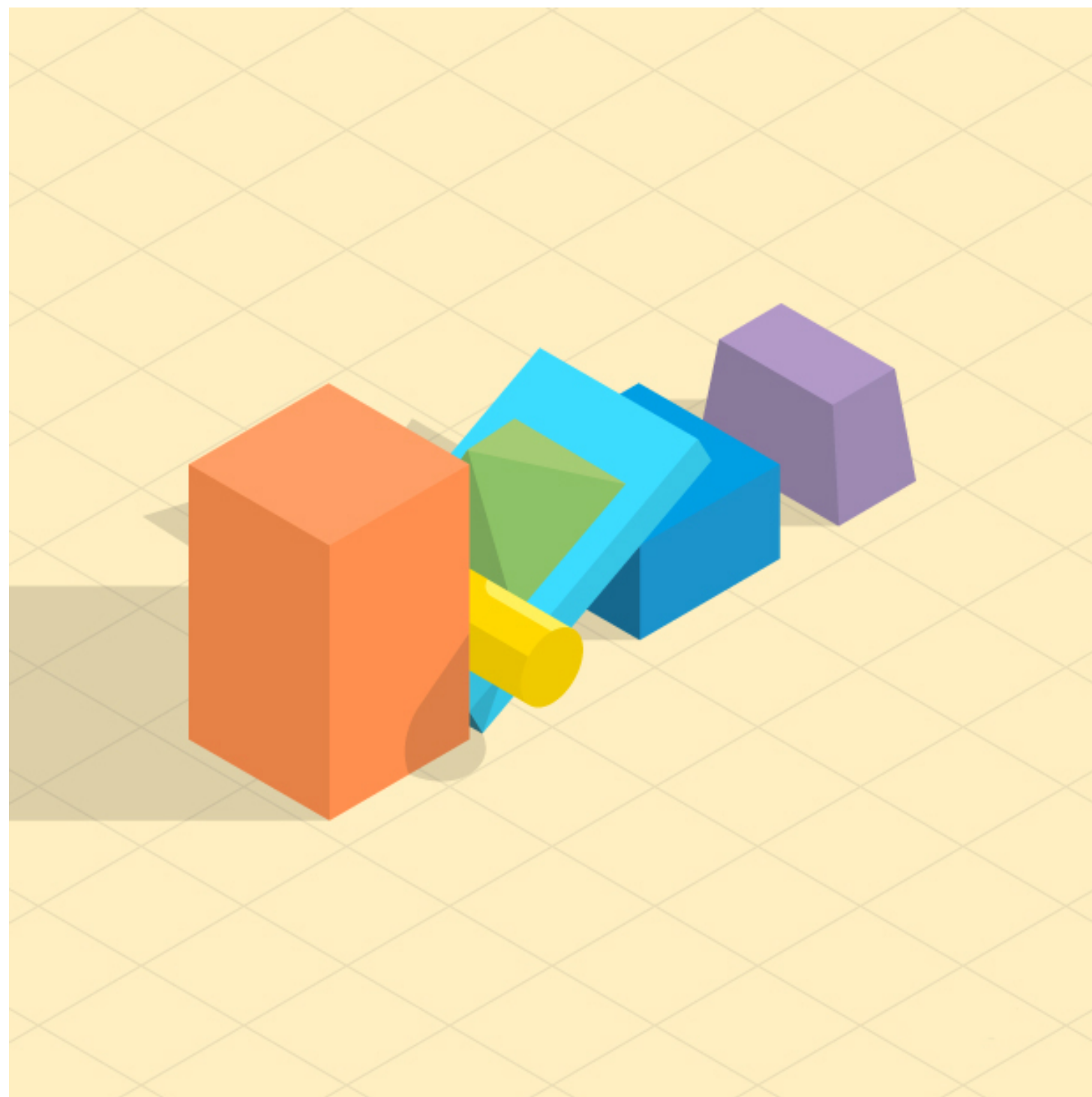
Por exemplo:

- ▶ Andar/Correr
- ▶ Pular
- ▶ Deslizar
- ▶ Planar
- ▶ Vento
- ▶ ...

Objetos Rígidos



Geralmente, os jogos que possuem física, assumem que os *game objects* são **Objetos Rígidos**, ou seja, são sólidos que não sofrem deformação. Suas propriedades são:



- ▶ **Massa** (escalar): quantidade de matéria no corpo
- ▶ **Posição** (vetor): localização no espaço (2D ou 3D)
- ▶ **Velocidade** (vetor): taxa de variação de posição
- ▶ **Acceleração** (vetor): taxa de variação de velocidade

Apesar de objetos rígidos não existirem na vida real, são excelentes simplificações para simulações de objetos em jogos.

Implementando Objetos Rígidos



Uma boa forma de implementar Objetos Rígidos, é utilizando um componente que pode ser adicionado à lista de componentes dos objetos que terão movimento do jogo:

```
class RigidBody : public Component
{
public:
    RigidBody(class Actor* owner, float mass);
    void ApplyForce(const Vector2 &force);
    void Update(float deltaTime) override;

private:
    // Physical properties
    float mMass;
    Vector2 mVelocity;
    Vector2 mAcceleration;
};
```

```
void ApplyForce(const Vector2 &force) {
    // Aplicar força no objeto
}
```

```
void Update(const deltaTime) {
    // Atualizar a posição do actor com base
    nas forças aplicadas
}
```

1. Como acumular as forças aplicadas?
2. Como aplicar essas forças para mover o objeto?

Movimentação de objetos rígidos



As duas perguntas são respondidas pelas leis da Física Newtoniana:

Primeira Lei de Newton:

Um objeto em repouso permanece em repouso, ou se estiver em movimento, permanece em movimento com velocidade constante, a menos que uma força externa atue sobre ele.

```
void Update(const deltaTime) {  
    mVelocity += mAcceleration * dt;  
    mPosition += mVelocity * dt;  
    mAcceleration.Set(.0, .0);  
}
```

Segunda Lei de Newton:

Força ($\vec{f}$) é igual a massa m vezes a aceleração $\vec{a}$: $\vec{f} = m\vec{a}$

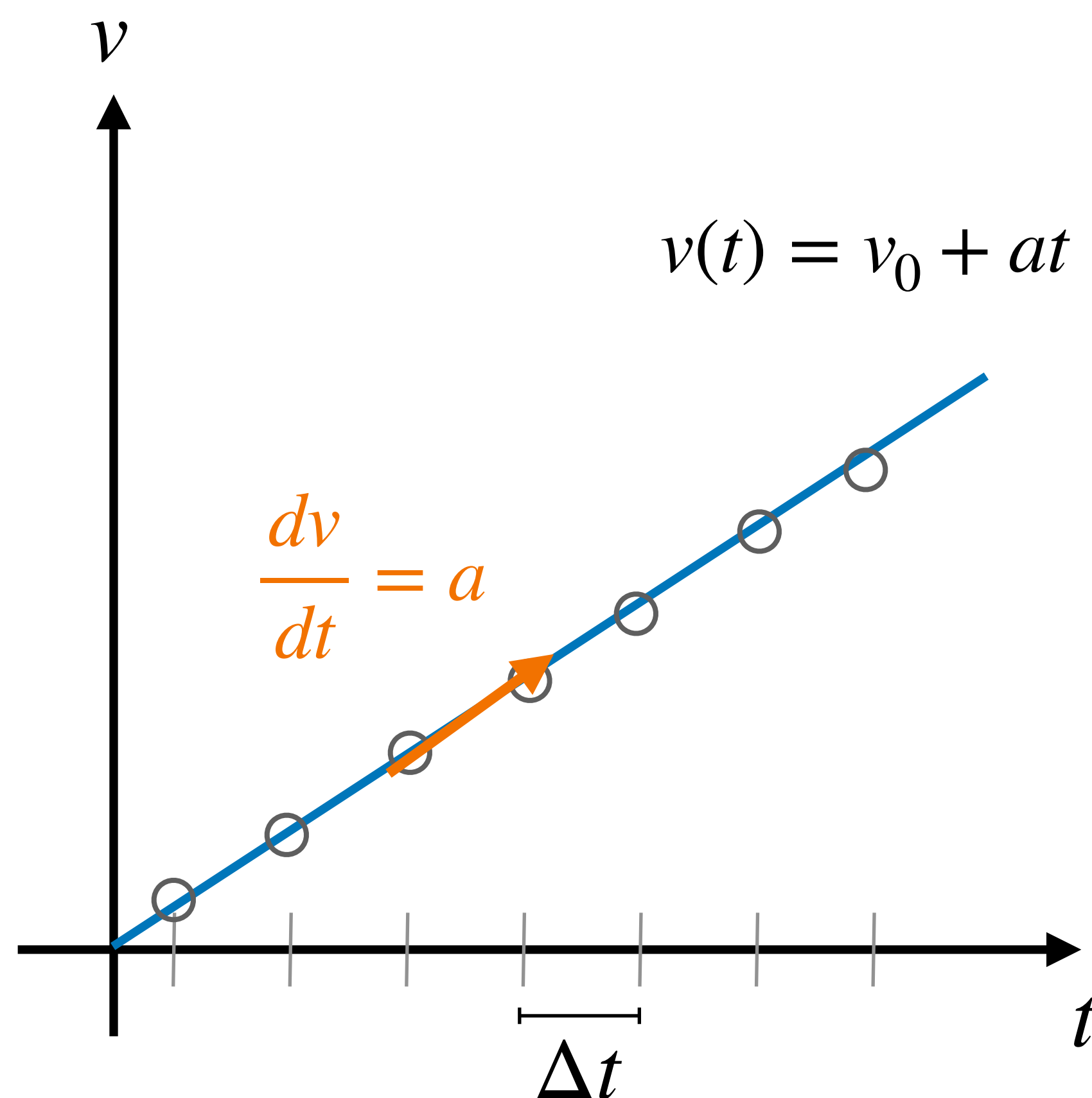
```
void ApplyForce(const Vector2 &force) {  
    mAcceleration += force * (1.f/mMass);  
}
```

Formalmente, essa solução é um método numérico de integração chamado de **Método de Euler Semi-Implicito**

Método de Euler Semi-Implicito



Assumindo que o jogo é um loop com FPS fixo, nosso objetivo é aproximar $v(t)$ e $x(t)$ em instantes discretos de tempo, separados por um intervalo fixo Δt :



Derivada aproximada considerando dois quadros consecutivos t e $t + \Delta t$:

$$\frac{dv}{dt} \approx \frac{v(t + \Delta t) - v(t)}{\Delta t}$$

Isolando $v(t + \Delta t)$:

$$v(t + \Delta t) \approx v(t) + \frac{dv}{dt} \cdot \Delta t$$

$\frac{dv}{dt}$ = aceleração a :

$$v(t + \Delta t) \approx v(t) + a \cdot \Delta t$$

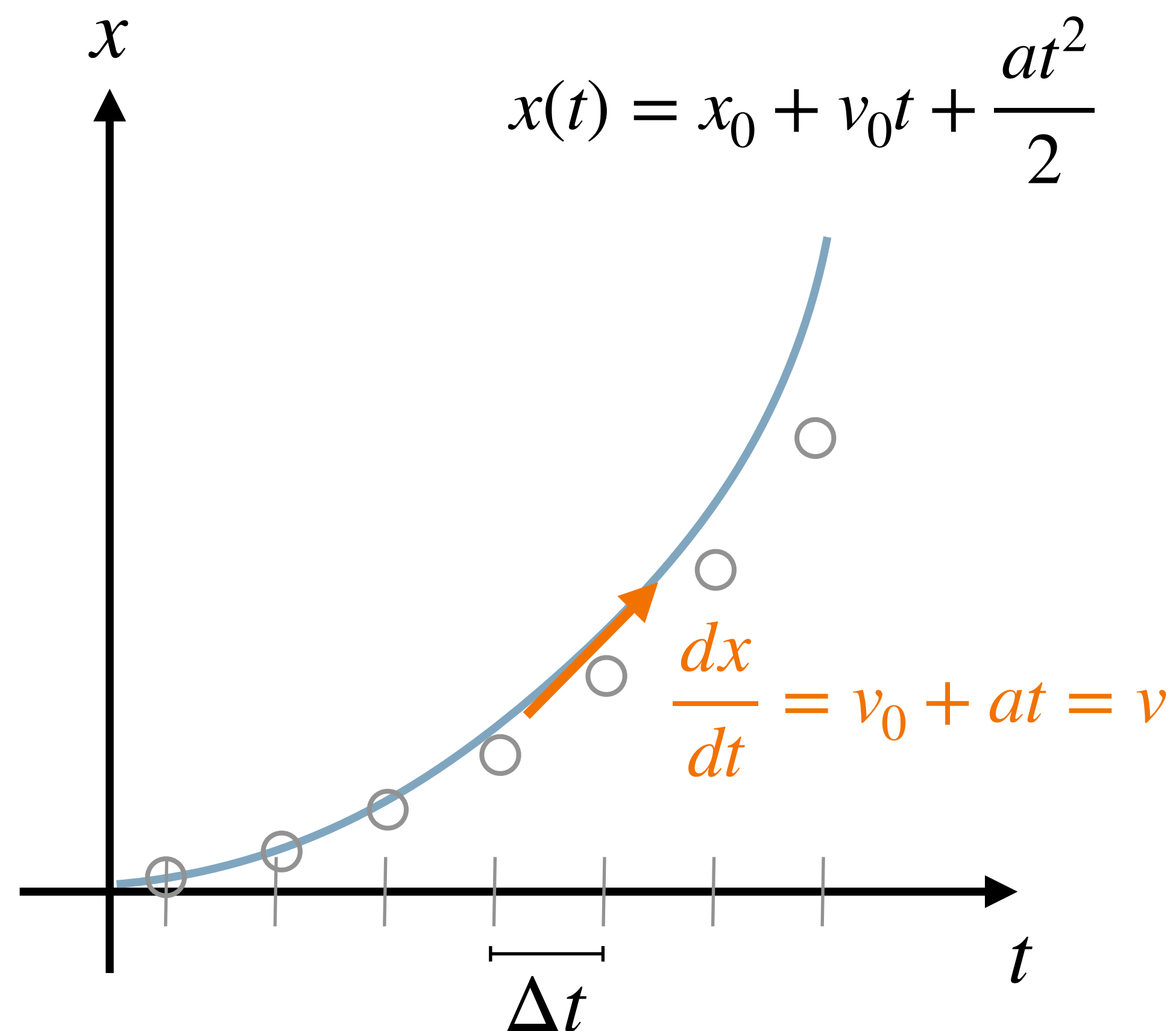
Renomeando as variáveis:

$$v' \leftarrow v + a \cdot \Delta t$$

Método de Euler Semi-Implicito



Assumindo que o jogo é um loop com FPS fixo, nosso objetivo é aproximar $v(t)$ e $x(t)$ em instantes discretos de tempo, separados por um intervalo fixo Δt :



Derivada aproximada considerando dois quadros consecutivos t e $t + \Delta t$:

$$\frac{dx}{dt} \approx \frac{x(t + \Delta t) - x(t)}{\Delta t}$$

Isolando $v(t + \Delta t)$:

$$x(t + \Delta t) \approx x(t) + \frac{dx}{dt} \cdot \Delta t$$

$\frac{dv}{dt} = \text{aceleração } a$:

$$x(t + \Delta t) \approx x(t) + v \cdot \Delta t$$

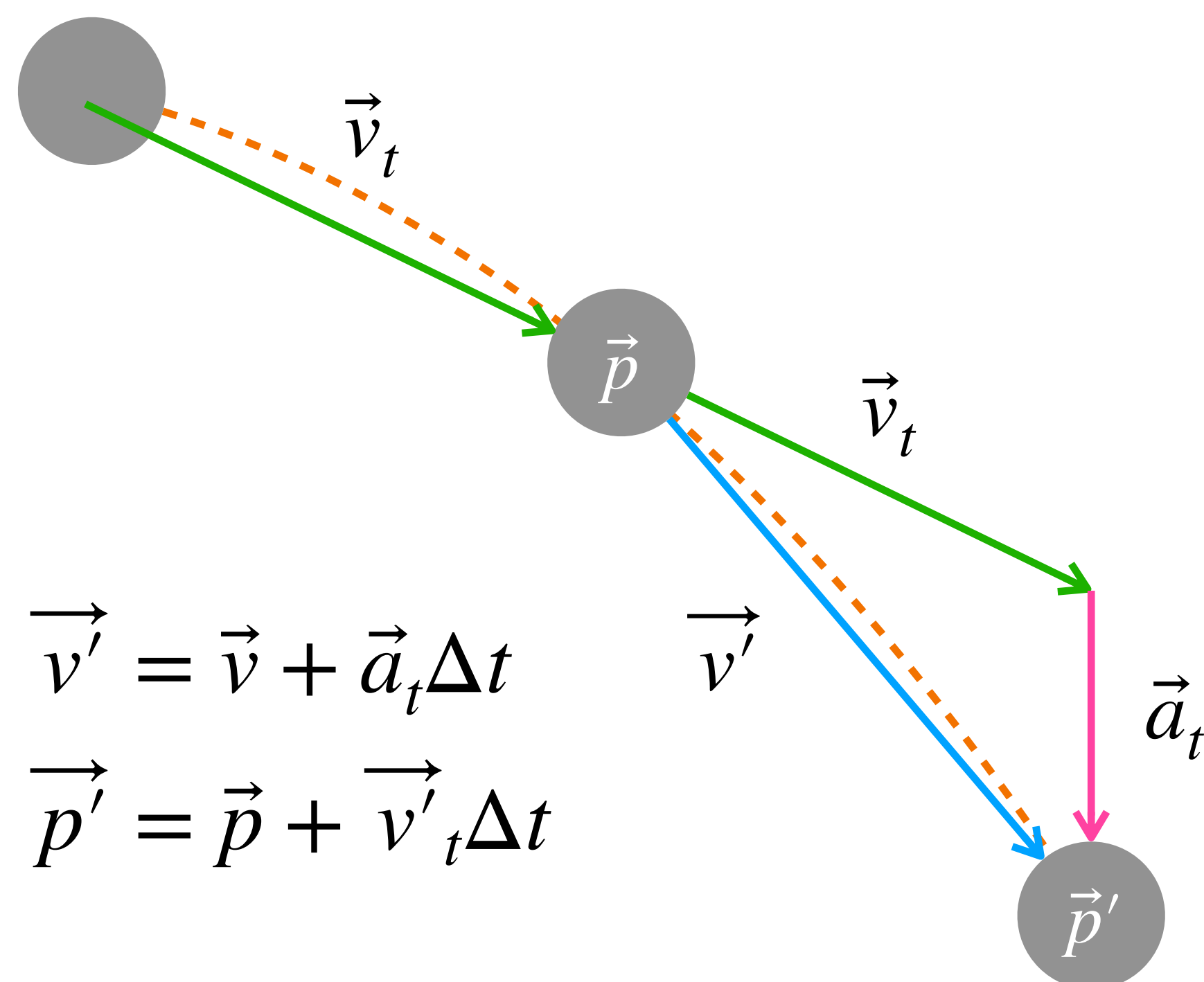
Renomeando as variáveis:

$$x' \leftarrow x + v \cdot \Delta t$$

Método de Euler Semi-Implicito



Geometricamente, as curvas dos movimentos contínuos reais são aproximados por uma sequência de retas:



$$\vec{v}' = \vec{v} + \vec{a}_t \Delta t$$

$$\vec{p}' = \vec{p} + \vec{v}'_t \Delta t$$

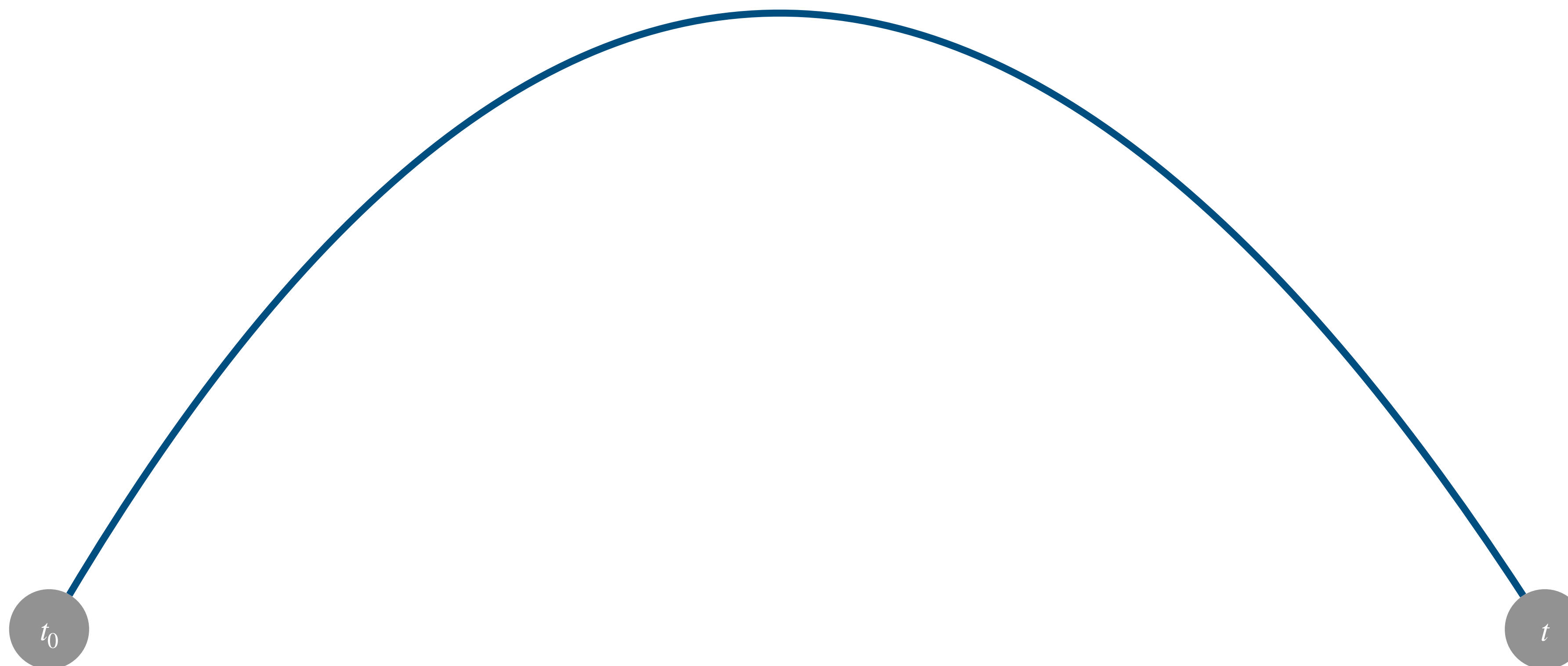
```
void Update(const deltaTime) {  
    mVelocity += mAcceleration * dt;  
    mPosition += mVelocity * dt;  
    mAcceleration.Set(.0, .0);  
}
```

- ▶ O intervalo de tempo Δt define a magnitude da aceleração $\vec{a}_t$
- ▶ Quanto menor o Δt , melhor a aproximação do **movimento real**.

Impacto do tamanho do delta time



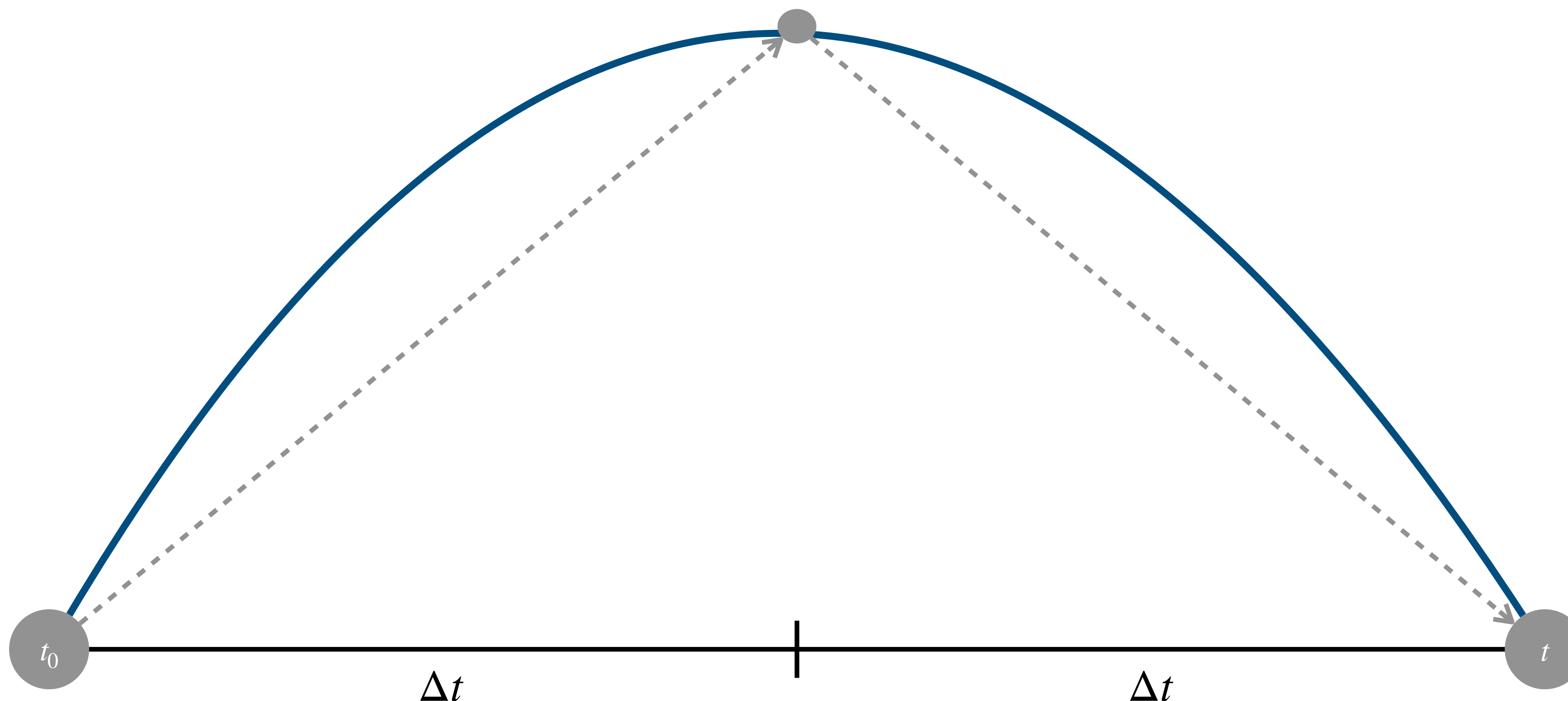
Quanto menor o Δt , melhor a aproximação do **movimento real**, ou seja, mais suave será o movimento.



Impacto do tamanho do delta time



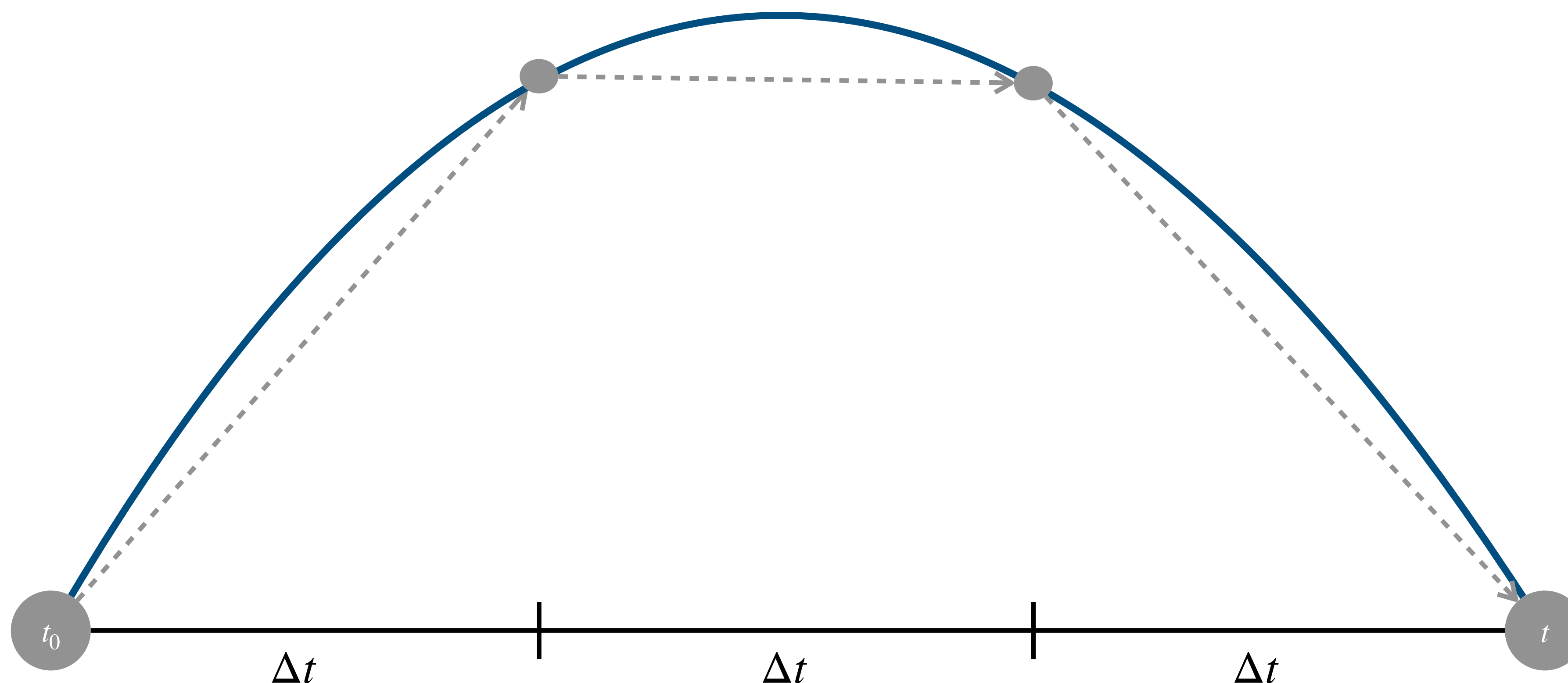
Quanto menor o Δt , melhor a aproximação do **movimento real**, ou seja, mais suave será o movimento.



Impacto do tamanho do delta time



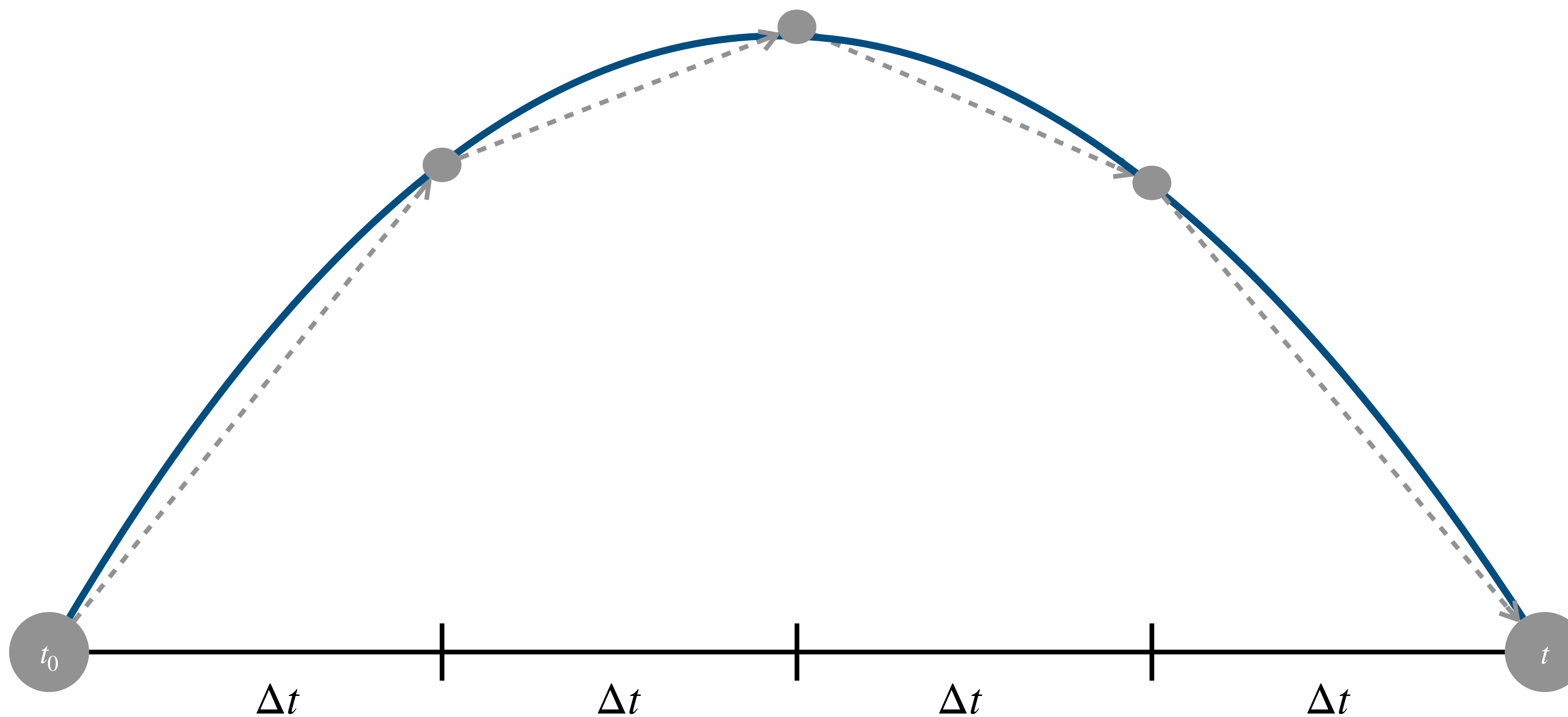
Quanto menor o Δt , melhor a aproximação do **movimento real**, ou seja, mais suave será o movimento.



Impacto do tamanho do delta time



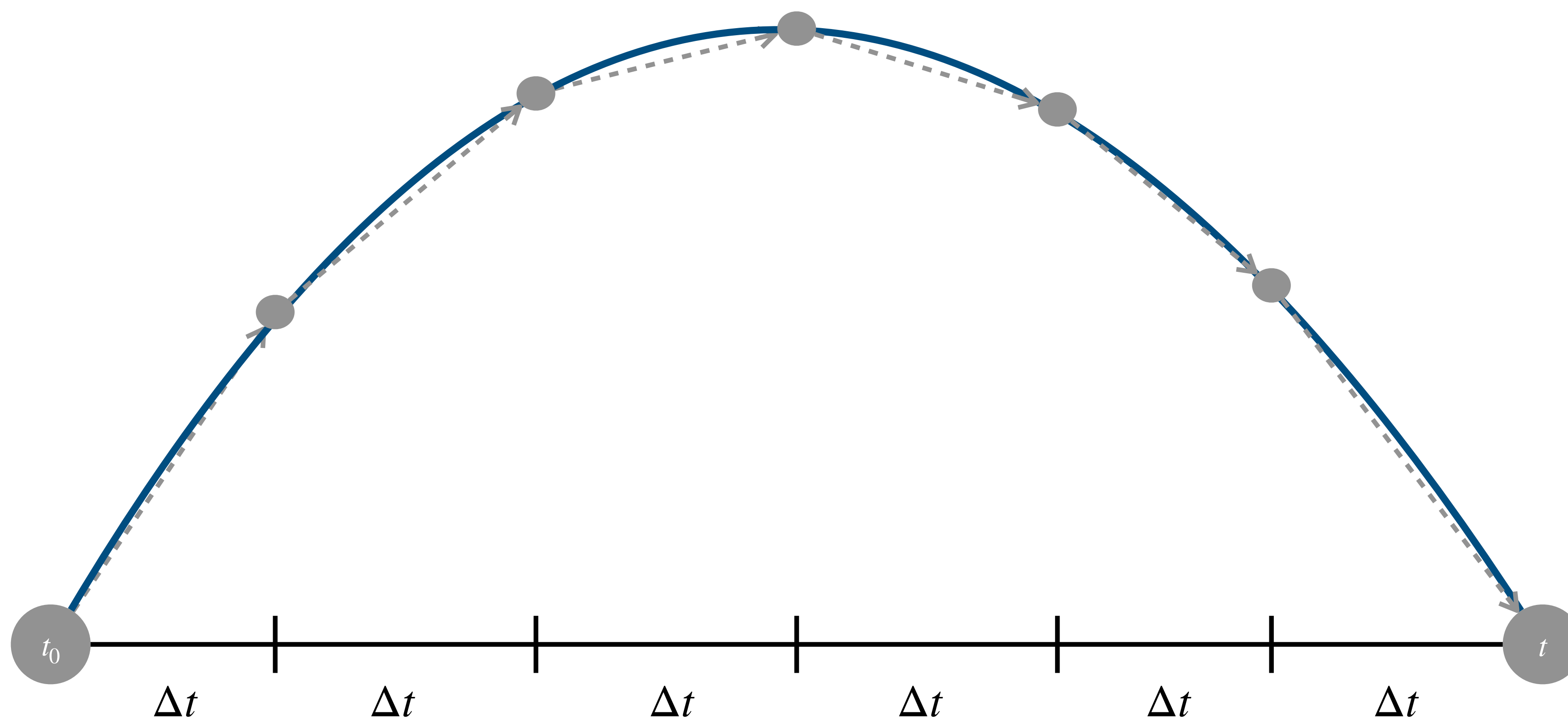
Quanto menor o Δt , melhor a aproximação do **movimento real**, ou seja, mais suave será o movimento.



Impacto do tamanho do delta time



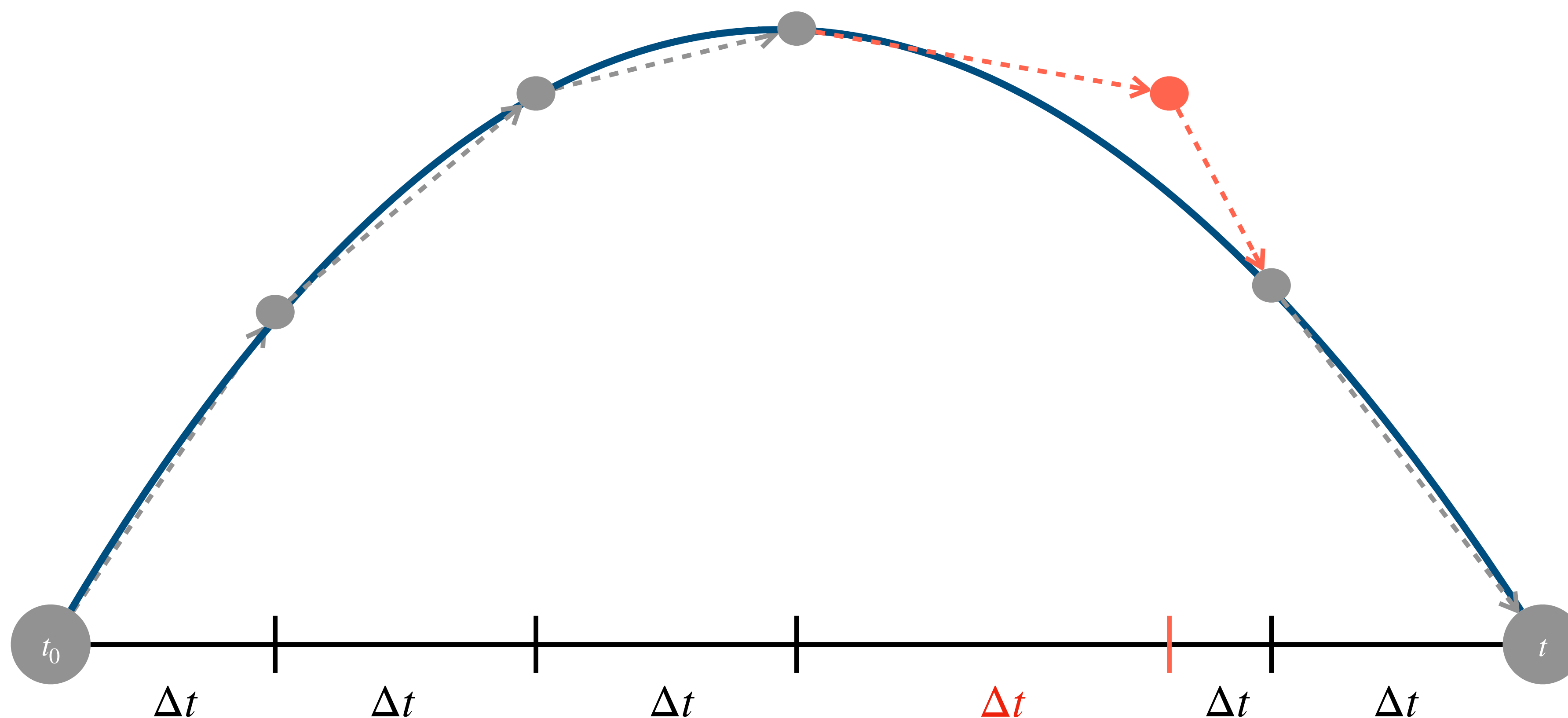
Quanto menor o Δt , melhor a aproximação do **movimento real**, ou seja, mais suave será o movimento.



Impacto na variação do delta time



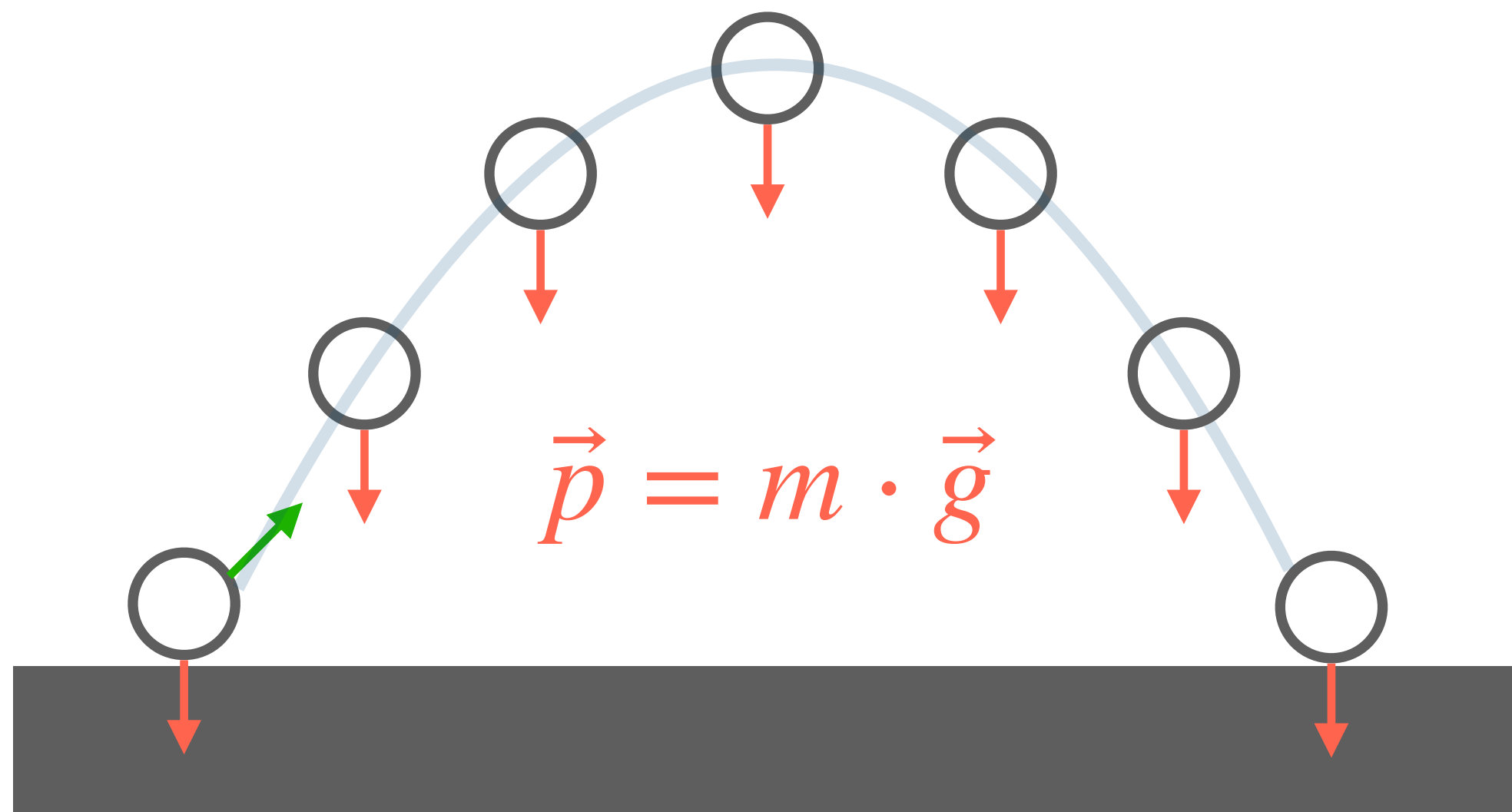
Como mencionados em aulas anteriores, se o delta time Δt for variável entre quadros, a simulação física pode ficar instável!



Aceleração da Gravidade



É muito comum jogos aplicarem uma **força peso** aos seus objetos, que é causada pela aceleração da gravidade.



```
Vector2 g = Vector2(0.f, 9.8f);
```

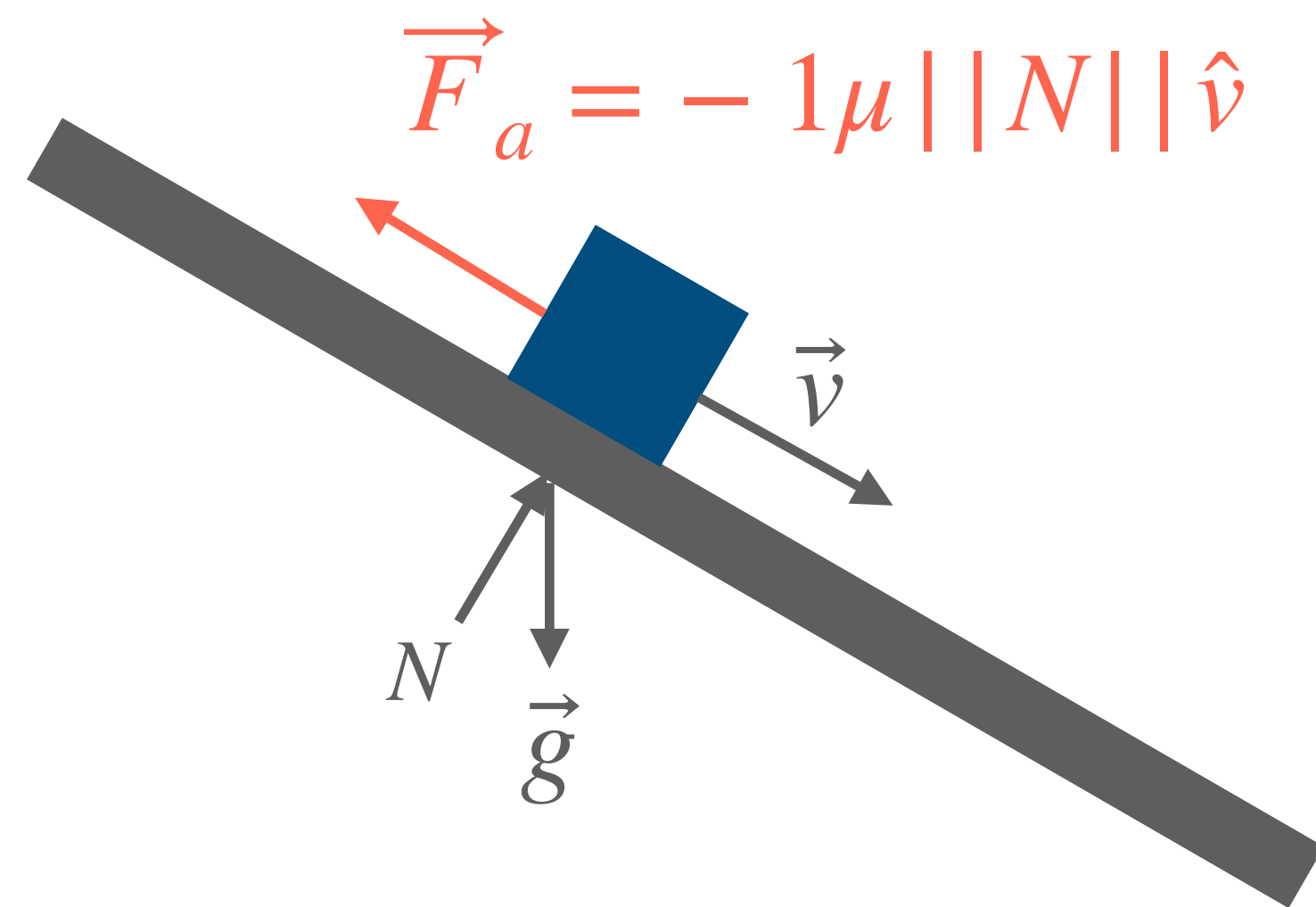
```
RigidBody::Update(float dt) {  
    ApplyForce(mMass * g);  
    mVelocity += mAcceleration * dt;  
    mPosition += position * dt;  
    mAcceleration.Set(0f, 0f);  
}
```

```
RigidBody::ApplyForce(Vector2 f) {  
    mAcceleration += f * 1f/mMass;  
}
```

Atrito



Também é muito comum implementar uma **força de atrito**, para parar um objeto quando outras forças não estão mais atuando.



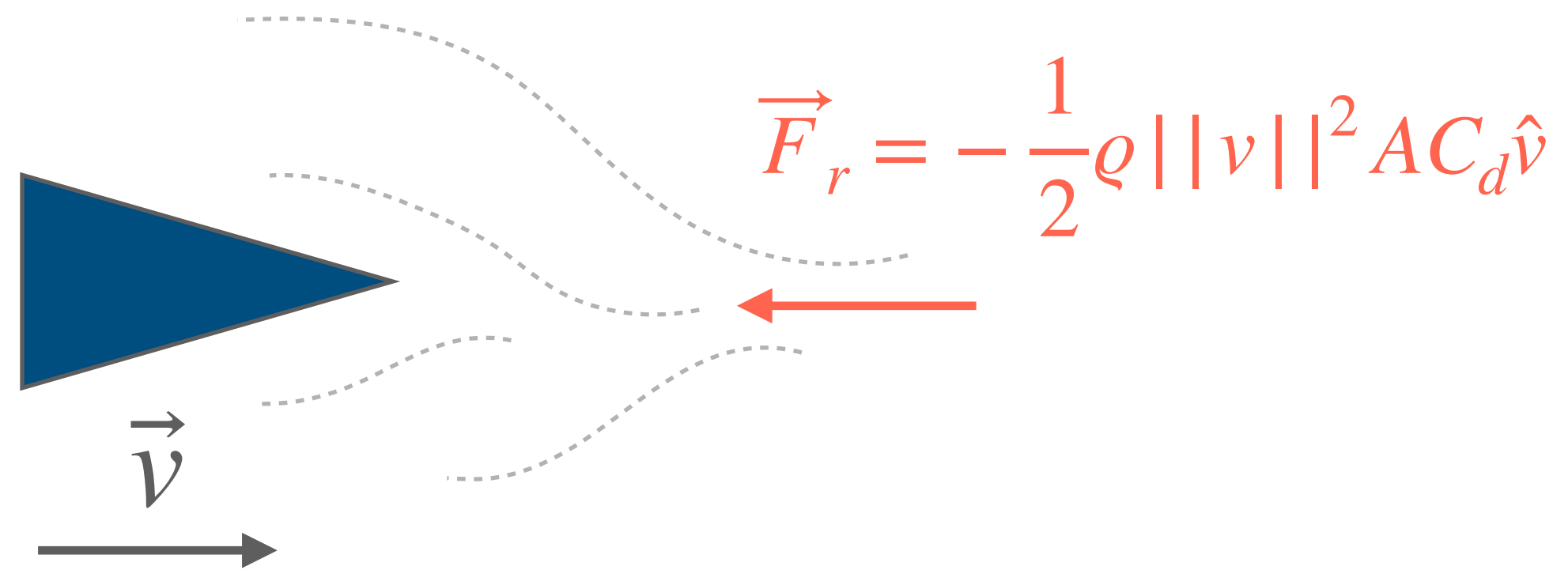
- ▶ μ : coeficiente de atrito
- ▶ N : é a força normal

```
RigidBody::ApplyFriction(float u, Vector2 N) {  
    if(mVelocity.Length() <= a)  
        return;  
  
    float frictionMag = u * N.Length();  
    Vector2 friction = (-1) * mVelocity;  
    friction.Normalize();  
    friction *= frictionMag;  
  
    ApplyForce(friction);  
}
```


Resistência do meio



A mesma ideia se aplica para parar um objeto que não está em contato com uma superfície, mas sobre **resistência do meio**, como do ar ou de um fluido.



- ▶ ρ : densidade do meio
- ▶ $||v||^2$: comprimento do vetor velocidade
- ▶ A : área frontal do objeto
- ▶ C_d : coeficiente de resistência
- ▶ $\hat{v}$: direção do vetor velocidade

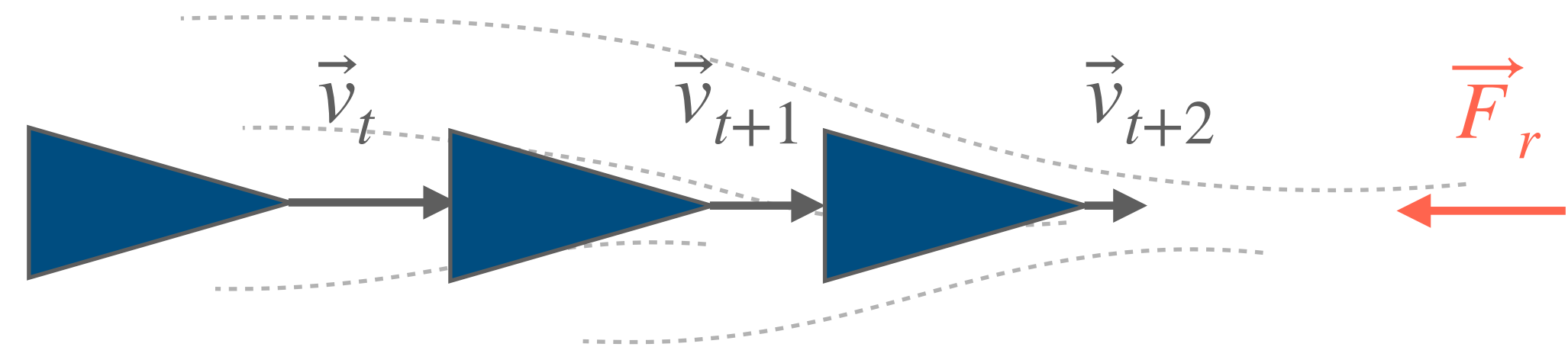
```
RigidBody::ApplyDrag(float rho) {  
    if(mVelocity.Length() <= MIN_VEL)  
        return;  
  
    float vLen = mVelocity.Length();  
    float dragLen = rho * vLen * vLen;  
  
    Vector2 drag = (-1) * mVelocity;  
    drag.Normalize();  
    drag *= dragLen;  
  
    ApplyForce(drag);  
}
```

Parando por Completo



Note que a aceleração é zerada a cada quadro, mas e a **velocidade não**, ou seja, o objeto só irá parar por completo quando a aceleração cancelar perfeitamente a velocidade atual:

```
RigidBody::Update(float dt) {  
    ApplyDrag(0.1f);  
    mVelocity += mAcceleration * dt;  
    mPosition += position * dt;  
    mAcceleration.Set(0f, 0f);  
}
```



Problemas:

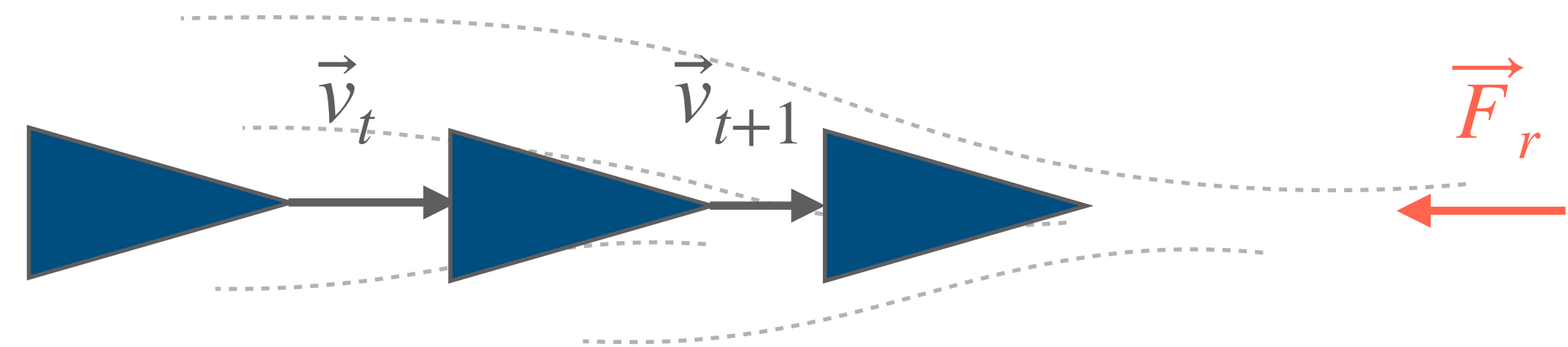
- ▶ $\vec{v} = (0,0)$ é extremamente improvável devido às multiplicações por dt e pela natureza de ponto flutuante das forças
- ▶ O objeto vai chegar a uma velocidade escalar muito baixa (ex. 0.001), mas nunca irá parar por completo

Parando por Completo



Note que a aceleração é zerada a cada quadro, mas e a **velocidade não**, ou seja, o objeto só irá parar por completo quando a aceleração cancelar perfeitamente a velocidade atual:

```
RigidBody::Update(float dt) {  
    ApplyDrag(0.1f);  
    mVelocity += mAcceleration * dt;  
  
    // Verificar velocidade mínima  
    if(mVelocity.Length() < MIN_VEL) {  
        mVelocity.Set(.0f, .0f);  
    }  
  
    mPosition += position * dt;  
    mAcceleration.Set(0f, 0f);  
}
```



Solução:

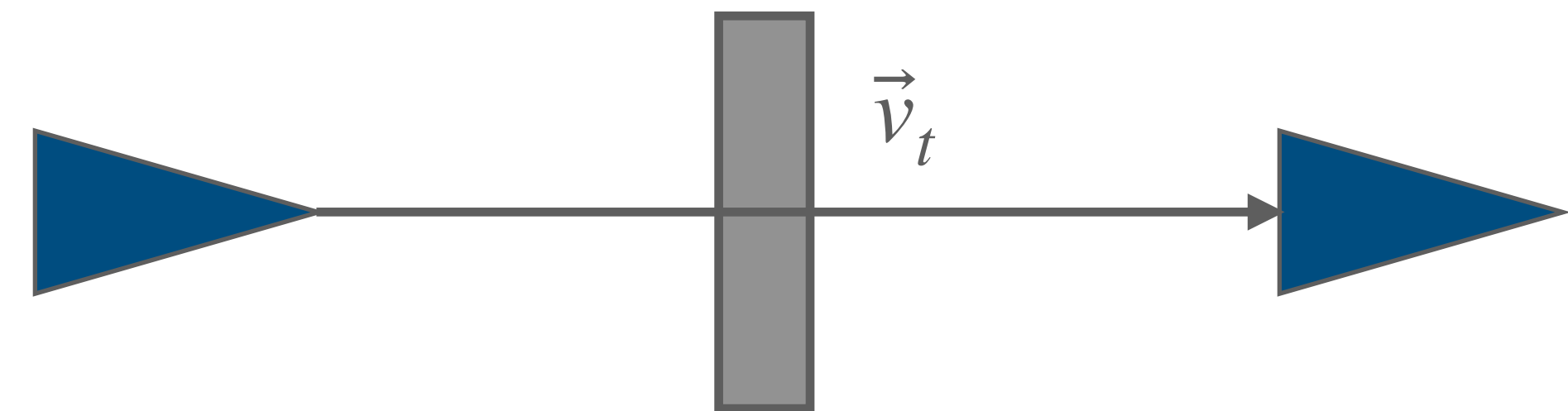
- ▶ Definir um limiar de velocidade escalar mínima `MIN_VEL`
- ▶ Se a velocidade escalar for menor que esse limiar, force ela a ser exatamente zero

Velocidade Máxima



Também é importante definir um limiar de **velocidade máxima**, para tornar a simulação mais controlada e evitar comportamentos inesperados (ex. atravessar paredes)

```
RigidBody::Update(float dt) {  
    mVelocity += mAcceleration * dt;  
    // Verificar velocidade mínima  
    ...  
    // Verificar velocidade máxima  
    if(mVelocity.Length() > MAX_VEL) {  
        mVelocity.Normalize();  
        mVelocity *= MAX_VEL;  
    }  
  
    mPosition += position * dt;  
    mAcceleration.Set(0f, 0f);  
}
```



Solução:

- ▶ Definir um limiar de velocidade escalar máxima MAX_VEL
- ▶ Se a velocidade escalar for maior que esse limiar, force ela a ser exatamente MAX_VEL

Próxima aula



A8: Física II – Detecção de Colisão

- ▶ Geometrias de colisão
 - ▶ Falsos Positivos
- ▶ Detecção de colisão
 - ▶ Circunferência vs. Circunferência
 - ▶ AABB vs. AABB
- ▶ Resolução de Colisão
 - ▶ AABB vs. AABB